

# **Algorithms**

## **Lecture # 31**

**Syed Hamed Raza**

# DFS-Time Stamp Structure

## ***DFS - Timestamp Structure***

- As we traverse the graph in DFS order, we will associate two numbers with each vertex.
- When we first discover a vertex  $u$ , store a counter in  $d[u]$ .
- When we are finished processing a vertex, we store a counter in  $f[u]$ .
- These two numbers are *time stamps*.

# DFS-Time Stamp Structure

## ***DFS - Timestamp Structure***

Consider the recursive version of  
depth-first traversal

DFS(G)

```
1  for (each  $u \in V$ )
2  do color[u]  $\leftarrow$  white
3     pred[u]  $\leftarrow$  nil
4  time  $\leftarrow$  0
5  for each  $u \in V$ 
6  do if (color[u] = white)
7     then DFSVISIT(u)
```

# DFS-Time Stamp Structure

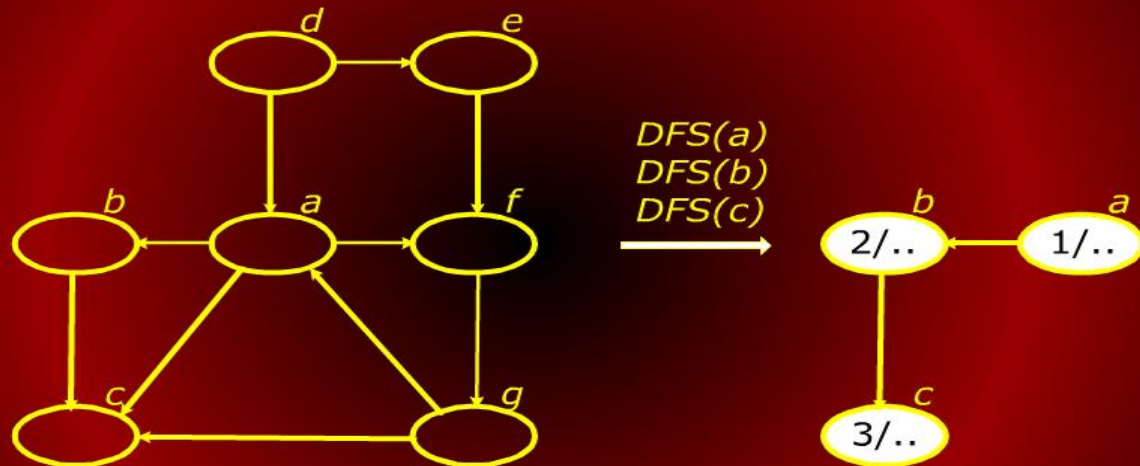
## ***DFS - Timestamp Structure***

DFSVISIT(u)

```
1  color[u] ← gray; //mark u visited
2  d[u] ← ++ time
3  for (each v ∈ Adj[u])
4  do if (color[v] = white)
5      then pred[v] ← u
6      DFSVISIT(v)
7  color[u] ← black; //we are done with u
8  f[u] ← ++ time;
```

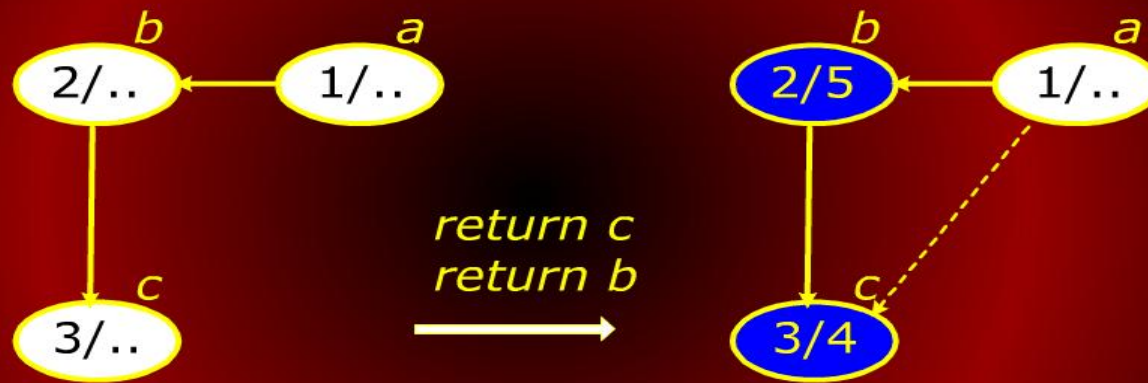
# DFS-Time Stamp Structure

## *DFS - Timestamp Structure*



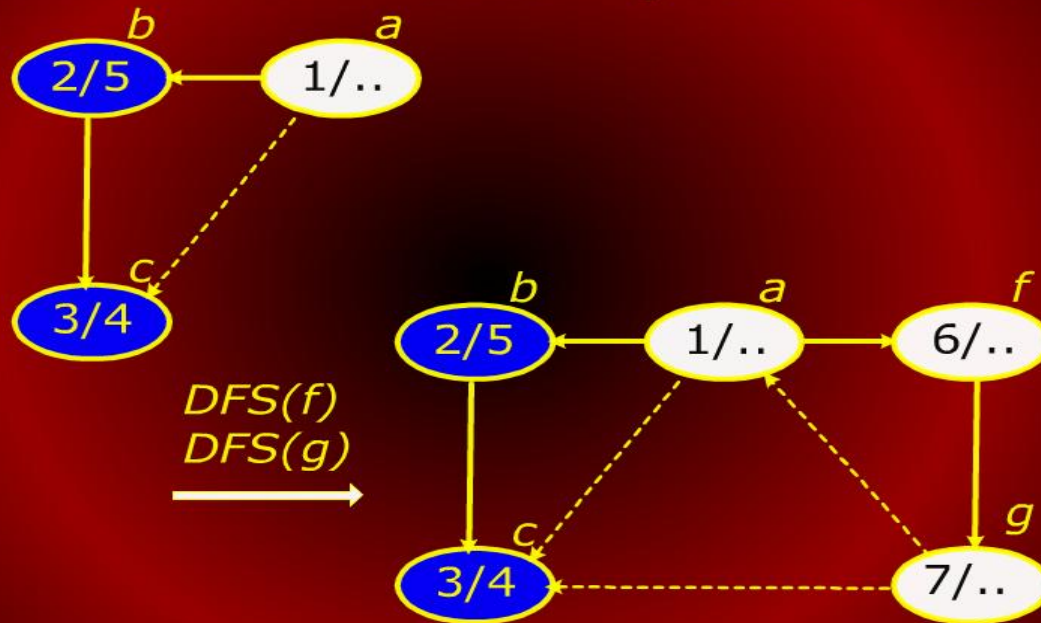
# DFS-Time Stamp Structure

## *DFS - Timestamp Structure*



# DFS-Time Stamp Structure

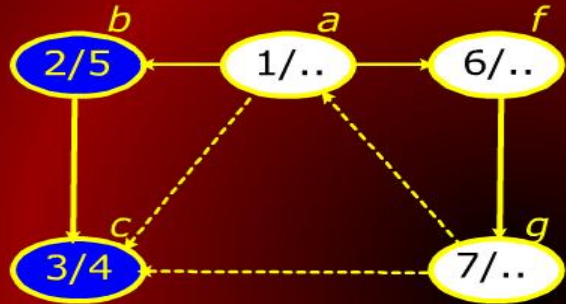
## *DFS - Timestamp Structure*





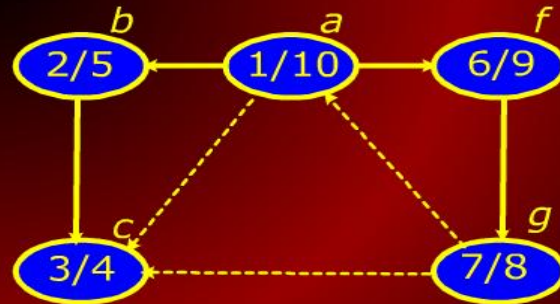
# DFS-Time Stamp Structure

## DFS - Timestamp Structure



return  $g$   
return  $f$   
return  $a$

A horizontal arrow pointing to the right, indicating the sequence of return operations.

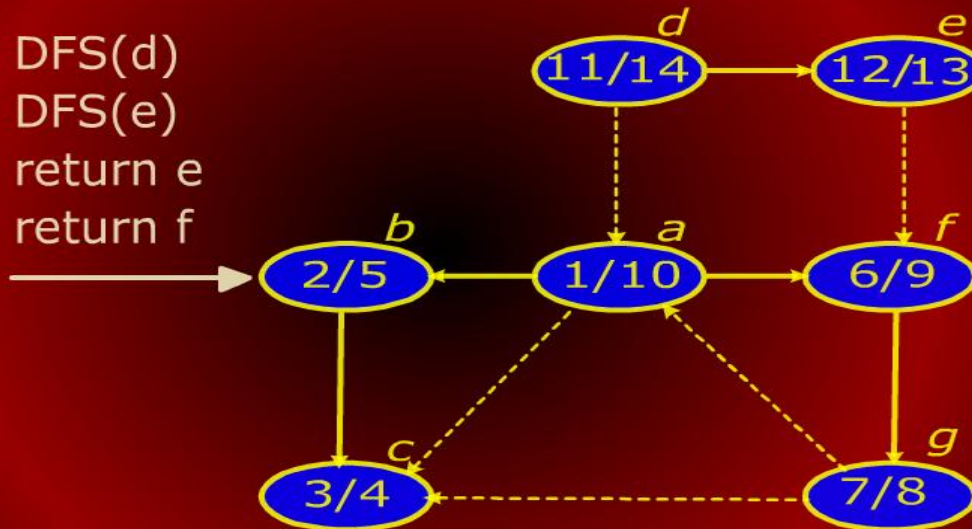




# DFS-Time Stamp Structure

## ***DFS - Timestamp Structure***

DFS(d)  
DFS(e)  
return e  
return f



# DFS-Tree Structure

## ***DFS - Tree Structure***

For a directed graph, the other edges (non-tree) can be classified as follows:

**Back edge:**  $(u, v)$  where  $v$  is an ancestor of  $u$  in the tree.

**Forward edge:**  $(u, v)$  where  $v$  is a proper descendent of  $u$  in the tree.

**Cross edge:**  $(u, v)$  where  $u$  and  $v$  are not ancestor or descendent of

# DFS-Tree Structure

## ***DFS - Tree Structure***

**Back edge:**  $(u, v)$  where  $v$  is an ancestor of  $u$  in the tree.

**Forward edge:**  $(u, v)$  where  $v$  is a proper descendent of  $u$  in the tree.

**Cross edge:**  $(u, v)$  where  $u$  and  $v$  are not ancestor or descendent of one another. In fact, the edge may go between different trees of the forest.

# DFS-Paranthesis Structure

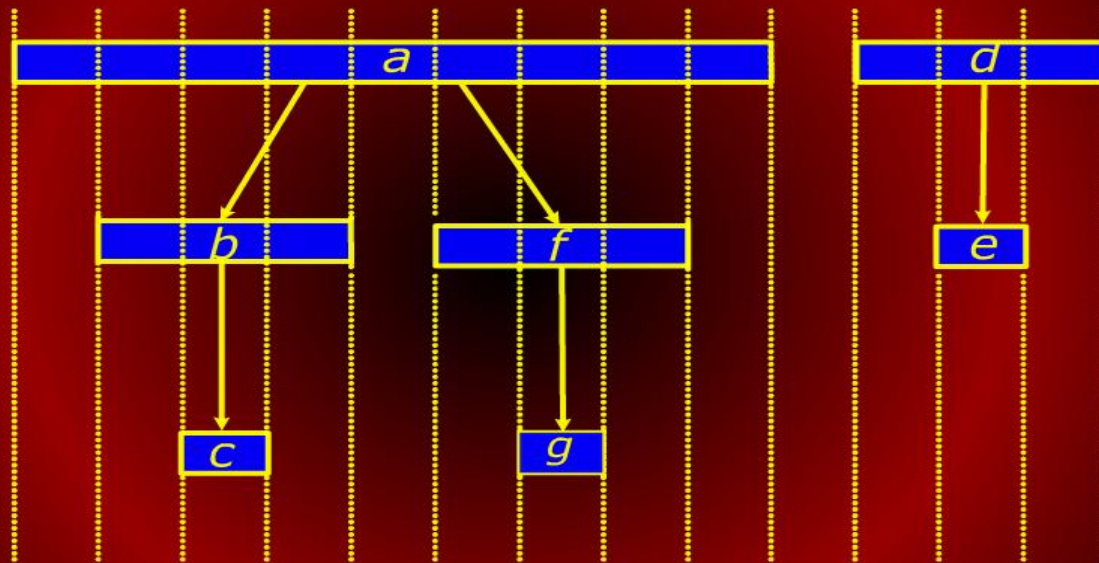
## ***DFS - Paranthesis Structure***

The ancestor and descendent relation can be nicely inferred by the parenthesis lemma.

- $u$  is a descendent of  $v$  if and only if  $[d[u], f[u]] \subseteq [d[v], f[v]]$ .
- $u$  is a ancestor of  $v$  if and only if  $[d[u], f[u]] \supseteq [d[v], f[v]]$ .
- $u$  is unrelated to  $v$  if and only if  $[d[u], f[u]]$  and  $[d[v], f[v]]$  are disjoint.

# DFS-Time Stamp Structure

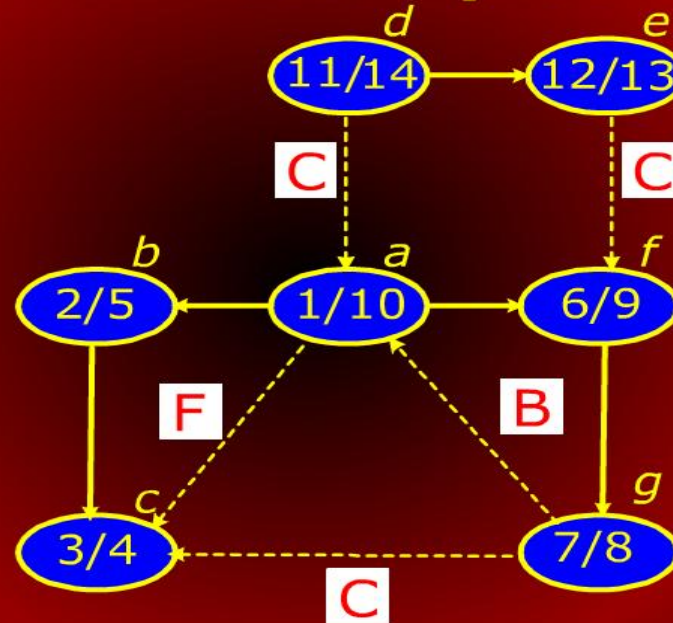
## *DFS - Timestamp Structure*





# DFS-Time Stamp Structure

## *DFS - Timestamp Structure*



# DFS

## Time Stamp Structure